

Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Sistemas Operativos 1

Sección P

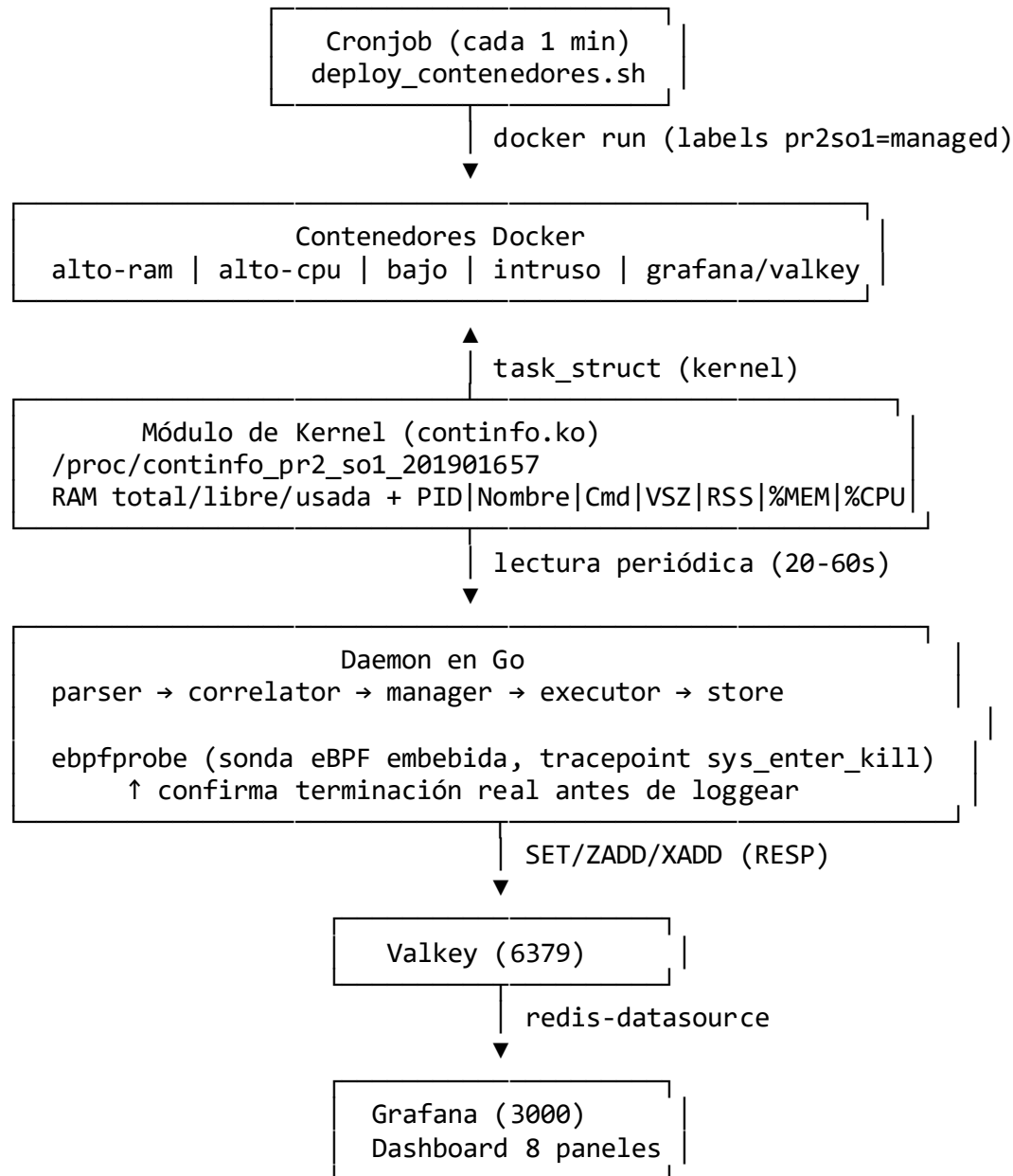
Sonda de Kernel en C y Daemon en Go para la Telemetría de Contenedores

Carlos Alejandro Rosales Medina

201901657

Guatemala, 19 de septiembre de 2026

1. Arquitectura



2. Componentes en detalle

2.1 Módulo de kernel

- Crea el archivo `/proc/continfo_pr2_so1_201901657` con permisos de solo lectura (0444).
- Usa `si_meminfo()` para obtener RAM total/libre/usada del sistema.
- Itera todos los procesos del sistema con `for_each_process()` sobre la lista de `task_struct`, protegido por `rcu_read_lock()`.
- Por cada proceso extrae:
 - PID y nombre (`task->comm`).
 - Línea de comandos: réplica manual de `get_cmdline()` (no exportada para módulos externos), leyendo `mm->arg_start/arg_end` vía `access_process_vm()`. Si el proceso no tiene `mm` (kernel thread) o no tiene argumentos visibles, se usa `[comm]` o `[kernel-thread:comm]` como fallback.
 - VSZ/RSS en KB, calculados desde `mm->total_vm` y `get_mm_rss(mm)`.
 - %MEM: `RSS / RAM_total`.
 - %CPU: tiempo de CPU consumido (`utime + stime`) dividido entre el tiempo de vida del propio proceso (no el uptime del sistema completo) — así procesos de corta vida sí reflejan un %CPU representativo.
- Formato de salida pensado para parseo trivial en Go: `PID|NOMBRE|CMD|VSZ_KB|RSS_KB|%MEM|%CPU`.

Carga/descarga: el Makefile compila el módulo (`make all`), lo firma para Secure Boot (`make sign`, requiere clave MOK) y lo carga/descarga con `insmod/rmmod` (`make load / make unload`).

2.2 Cronjob y perfiles de carga

`deploy_contenedores.sh` corre cada 1 minuto y en cada ejecución:

- Lanza 5 contenedores elegidos aleatoriamente entre 3 perfiles:

Perfil	Imagen	Comando
alto-ram	roldyoran/go-client	(por defecto de la imagen)
alto-cpu	alpine	<code>sh -c "while true; do echo '2^1000000' bc > /dev/null; done"</code>
bajo	alpine	<code>sleep 240</code>

- Lanza 1 contenedor intruso (`intruso-pr2-201901657:latest`, `cronjob/intruso-image/Dockerfile`) que simula lectura de archivos sensibles (`/etc/passwd`, `/etc/shadow`, `~/.ssh/id_rsa`) dentro de su propio filesystem aislado — no representa una amenaza real al host.
- Todos los contenedores gestionables llevan las labels `pr2so1=managed` y `pr2so1-perfil=<perfil>`, que es como el Daemon los identifica y clasifica.

2.3 Daemon en Go

Estructurado en paquetes con responsabilidad única:

Paquete	Responsabilidad
parser	Lee y parsea /proc/continfo_pr2_so1_201901657 a structs Go (MemInfo, Process)
correlator	Asocia cada PID a su contenedor leyendo /proc/<pid>/cgroup (cgroup v1 y v2/systemd)
dockerinfo	Consulta docker ps para saber qué contenedores son gestionables y su perfil; puede lanzar (Launch) o matar (Kill, fallback) contenedores
manager	Agrega métricas por contenedor y decide qué eliminar / qué dar de alta
manager (executor.go)	Ejecuta las terminaciones (SIGTERM → espera de gracia → SIGKILL) y las altas
ebpfprobe	Carga la sonda eBPF embebida (go:embed) y expone confirmaciones reales de sys_kill
store	Persiste snapshots, rankings y contadores en Valkey

Ciclo principal:

1. Lee snapshot del /proc del kernel.
2. Correlaciona procesos → contenedores.
3. Guarda snapshot de RAM en Valkey (tarjetas + stream de series de tiempo).
4. Lista contenedores gestionables vía Docker.
5. Agrega métricas y actualiza los rankings históricos (Top RAM / Top CPU).
6. Decide terminaciones (manager.Decide) y altas (manager.DecideLaunches).
7. Ejecuta terminaciones enviando señales directamente a todos los PIDs del host que pertenecen a cada contenedor (no solo al PID “principal”, evitando heurísticas frágiles).
8. Espera hasta 5s la confirmación real de la sonda eBPF antes de registrar la eliminación en Valkey.
9. Ejecuta las altas necesarias vía Docker.

Reglas de decisión:

- Los contenedores de perfil intruso se eliminan siempre, sin excepción.
- Los contenedores de alto consumo (alto-ram, alto-cpu) se ordenan de mayor a menor consumo (RSS → VSZ → %CPU como criterios de desempate) y se eliminan los de mayor consumo mientras el conteo se mantenga por encima del mínimo de 2.
- Los contenedores de bajo consumo se ordenan igual y se eliminan mientras el conteo se mantenga por encima del mínimo de 3.
- El contenedor de Grafana nunca se gestiona porque no lleva el label pr2so1=managed — queda fuera del alcance del Daemon por diseño.

- Si algún perfil cae por debajo de su mínimo, DecideLaunches calcula cuántos hay que levantar y ExecuteLaunches los crea vía Docker con las mismas labels que el cronjob.

2.4 Sonda eBPF

- Se engancha al tracepoint syscalls:sys_enter_kill.
- Por cada llamada a kill() en el sistema, publica en un ring buffer (256 KB) un evento con: PID emisor, PID receptor y señal enviada.
- El Daemon en Go (ebpfprobe) lee este ring buffer en segundo plano y, cuando decide terminar un contenedor, espera hasta 5 segundos a que la sonda confirme que el/los PID(s) objetivo efectivamente recibieron la señal — esta es la “confirmación real a nivel de kernel” exigida por el enunciado antes de registrar el evento como “Contenedor Eliminado” en Valkey.
- Compilación separada con clang -target bpf (ver ebpf-sonda/Makefile); el objeto .o resultante se empotra en el binario del Daemon con go:embed.

2.5 Grafana + Valkey

- Valkey (valkey/valkey:latest) expone el puerto 6379, con volumen persistente pr2so1-valkey-data.
- Grafana (grafana/grafana:latest) expone el puerto 3000, con el plugin redis-datasource preinstalado (variable GF_PLUGINS_PREINSTALL) para poder leer Valkey nativamente (soporta XRange/XRevRange para streams y ZRange para sorted sets, pero no LRange — por eso el diseño de claves en store.go usa Streams y Sorted Sets en vez de listas).
- Ambos se levantan automáticamente al iniciar el Daemon (main.go → startGrafana()) ejecuta docker compose up -d).

Diseño de claves en Valkey:

Clave / Stream	Tipo	Uso en el dashboard
pr2so1:ram:total_kb	SET/GET	Tarjeta “Total RAM”
pr2so1:ram:free_kb	SET/GET	Tarjeta “Memoria Libre”
pr2so1:ram:used_kb	SET/GET	Tarjeta “RAM Usada”
pr2so1:ram:stream	XADD/XRANGE	“Uso de RAM a lo largo del tiempo” (time series)
pr2so1:eliminados:total	INCRBY/GET	Contador acumulado
pr2so1:eliminados:stream	XADD/XRANGE	“Contenedores Eliminados a lo largo del tiempo”
pr2so1:ranking:ram	ZADD GT	“Top 5 Contenedores por Consumo de RAM” (pie chart)
pr2so1:ranking:cpu	ZADD GT	“Top 5 Contenedores por Consumo de CPU” (pie chart)

Los rankings usan ZADD GT (solo sube el score, nunca baja).

El dashboard fue creado manualmente en la UI de Grafana y persiste en el volumen Docker pr2so1-grafana-data; incluye los 8 paneles requeridos: Total RAM, RAM Usada, Memoria Libre, Uso de RAM en el tiempo, Contenedores Eliminados en el tiempo, Top 5 RAM, Top 5 CPU y Eventos eBPF (contador de eliminaciones confirmadas por sys_kill).

3. Guía de instalación

3.1 Requisitos previos

- Linux con headers del kernel instalados (linux-headers-\$(uname -r)).
- gcc, make, clang (para compilar la sonda eBPF con -target bpf).
- Docker y Docker Compose.
- Go 1.23+.
- Clave MOK configurada para Secure Boot, si aplica (/var/lib/shim-signed/mok/), o Secure Boot deshabilitado.

3.2 Compilar el módulo de kernel

```
cd proyecto2/kernel-module
make # compila continfo.ko
make sign # firma el módulo (requiere clave MOK)
make load # sudo insmod continfo.ko
```

Verificar que el /proc se creó:

```
cat /proc/continfo_pr2_so1_201901657
```

Para descargarlo manualmente:

```
make unload # sudo rmmmod continfo
```

3.3 Compilar la sonda eBPF

```
cd proyecto2/ebpf-sonda
make # genera kill_monitor.o (usado vía go:embed por el Daemon)
```

3.4 Construir las imágenes de Docker necesarias

```
# Imagen intrusa
cd proyecto2/cronjob/intruso-image
docker build -t intruso-pr2-201901657:latest .
```

Las imágenes alpine y roldyoran/go-client se descargan automáticamente de Docker Hub la primera vez que se usan.

3.5 Compilar y ejecutar el Daemon

```
cd proyecto2/daemon-go
go build -o daemon-pr2-so1 .
sudo ./daemon-pr2-so1
```

Se requiere sudo porque: - La sonda eBPF necesita CAP_BPF/CAP_SYS_ADMIN para cargarse. - syscall.Kill() sobre PIDs de otros contenedores requiere privilegios de root. - Cargar el módulo de kernel (insmod) requiere privilegios de root.

Al iniciar, el Daemon automáticamente: 1. Levanta Grafana y Valkey (docker compose up -d). 2. Carga el módulo de kernel si no está cargado. 3. Instala el cronjob en crontab (cada 1 minuto). 4. Conecta a Valkey y verifica con PING. 5. Arranca la sonda eBPF. 6. Entra al loop principal (cada 30s).

3.6 Verificar el dashboard

Abrir <http://localhost:3000> (usuario/clave por defecto: admin/admin, definidos en GF_SECURITY_ADMIN_PASSWORD del docker-compose.yml) y confirmar que los 8 paneles muestran datos actualizándose.

3.7 Apagado ordenado

Ctrl+C (SIGINT) o kill (SIGTERM) sobre el proceso del Daemon dispara el apagado ordenado: remueve la entrada del cronjob de la crontab del sistema antes de finalizar, para no dejar contenedores generándose sin control.

4. Modificaciones acordadas respecto al enunciado original

Ítem	Enunciado original	Implementado (acordado)
Frecuencia del cronjob	Cada 2 minutos	Cada 1 minuto
Contenedores por ejecución	(no aplica/10 según interpretación previa)	5 aleatorios + 1 intruso

5. Evidencia funcional

5.1 Módulo de kernel cargado

```
lsmod  
cat /proc/continfo_pr2_so1_201901657
```

```
grep
```

```
continfo
```

```
alejandro@alejandro-HP-Laptop-15-da0xxx: ~/Escritorio/201901657_LAB_...
alejandro@alejandro-HP-Laptop-15-da0xxx: ~/Escritorio/201901657_LAB_S01_2S2026/pr
proyecto2/daemon-go$ lsmod | grep continfo
cat /proc/continfo_pr2_so1_201901657
continfo 16384 0
=== INFORMACION DE MEMORIA RAM ===
RAM_TOTAL_KB: 12157188
RAM_LIBRE_KB: 1317080
RAM_USADA_KB: 10840108

=== PROCESOS (PID|NOMBRE|CMD|VSZ_KB|RSS_KB|PORC_MEM|PORC_CPU) ===
1|systemd|/sbin/init splash|24864|15764|0.12|0.12
2|kthreadd|[kernel-thread:kthreadd]|0|0|0.00|0.00
3|pool_workqueue|[kernel-thread:pool_workqueue]|0|0|0.00|0.00
4|kworker/R-rcu_g|[kernel-thread:kworker/R-rcu_g]|0|0|0.00|0.00
5|kworker/R-sync|[kernel-thread:kworker/R-sync]|0|0|0.00|0.00
6|kworker/R-kvfre|[kernel-thread:kworker/R-kvfre]|0|0|0.00|0.00
7|kworker/R-slab|[kernel-thread:kworker/R-slab]|0|0|0.00|0.00
8|kworker/R-netns|[kernel-thread:kworker/R-netns]|0|0|0.00|0.00
10|kworker/0:0H|[kernel-thread:kworker/0:0H]|0|0|0.00|0.00
13|kworker/R-mm_pe|[kernel-thread:kworker/R-mm_pe]|0|0|0.00|0.00
14|ksoftirqd/0|[kernel-thread:ksoftirqd/0]|0|0|0.00|0.00
15|rcu_preempt|[kernel-thread:rcu_preempt]|0|0|0.00|0.00
16|rcu_exp_par_gp|[kernel-thread:rcu_exp_par_gp]|0|0|0.00|0.00
17|rcu_exp_gp_kthr|[kernel-thread:rcu_exp_gp_kthr]|0|0|0.00|0.00
```

5.2 Daemon corriendo — log de un ciclo completo

```
cd
sudo ./daemon-pr2-so1
```

proyecto2/daemon-go

```
alejandro@alejandro-HP-Laptop-15-da0xxx: ~/Escritorio/201901657_LAB_...
alejandro@alejandro-HP-Laptop-15-da0xxx: ~/Escritorio/201901657_LAB_S01_2S2026$ c
d proyecto2/daemon-go
sudo ./daemon-pr2-so1
[sudo] contraseña para alejandro:
2026/09/18 21:15:24 === Daemon Proyecto 2 S01 (201901657) iniciando ===
2026/09/18 21:15:26 Grafana (docker compose) levantado.
2026/09/18 21:15:26 Modulo de kernel cargado.
2026/09/18 21:15:26 Cronjob instalado (cada 1 minuto).
2026/09/18 21:15:26 Conexion a Valkey OK.
2026/09/18 21:15:26 Sonda eBPF activa (syscalls:sys_enter_kill).
2026/09/18 21:15:26 Daemon corriendo. Ciclo cada 30s. Ctrl+C para detener.
2026/09/18 21:15:26 --- Nuevo ciclo ---
Contenedores gestionables: 0
OK: nuevo contenedor perfil=alto-cpu lanzado
OK: nuevo contenedor perfil=alto-cpu lanzado
OK: nuevo contenedor perfil=bajo lanzado
OK: nuevo contenedor perfil=bajo lanzado
OK: nuevo contenedor perfil=bajo lanzado
Ciclo completo: 0 eliminados (confirmados por eBPF), 5 altas.
2026/09/18 21:15:35 [eBPF] sys_kill capturado: caller_pid=351 target_pid=9159 se
ñal=SIGTERM
2026/09/18 21:15:35 [eBPF] sys_kill capturado: caller_pid=351 target_pid=9161 se
ñal=SIGTERM
2026/09/18 21:15:35 [eBPF] sys_kill capturado: caller_pid=351 target_pid=9162 se
```

5.3 Dashboard de Grafana



Dashboard de Grafana con los 8 paneles: Total RAM, RAM Usada, Memoria Libre, Uso de RAM en el tiempo, Contenedores Eliminados en el tiempo, Top 5 por RAM, Top 5 por CPU y Eventos eBPF

5.4 docker ps — contenedores gestionados con sus labels

docker ps --filter "label=pr2so1=managed"

docker inspect --format '{{.Name}}: {{.Config.Labels}}' \$(docker ps --filter "label=pr2so1=managed" -q)

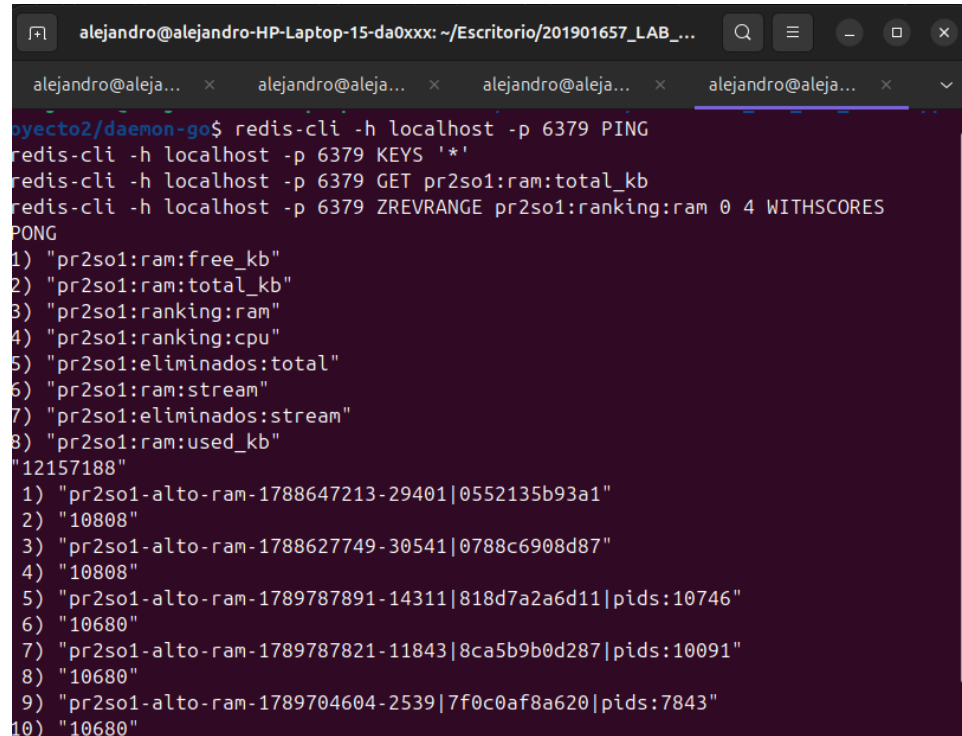
```

alejandro@alejandros-HP-Laptop-15-da0xxx: ~/Escritorio/201901657_LAB_...
alejandro@alejandros-HP-L... x alejandro@alejandros-HP-L... x alejandro@alejandros-HP-L... x
alejandro@alejandros-HP-Laptop-15-da0xxx: ~/Escritorio/201901657_LAB_S01_2S2026/pr
yecto2/daemon-go$ docker ps --filter "label=pr2so1=managed"
docker inspect --format '{{.Name}}: {{.Config.Labels}}' $(docker ps --filter "la
bel=pr2so1=managed" -q)
CONTAINER ID   IMAGE      COMMAND                  CREATED
STATUS        PORTS      NAMES
4ec5ba4e54a1   roldyoran/go-client  "/client"              16 seconds ago
Up 6 seconds   pr2so1-alto-ram-1789787922-16507
0035ca3174b0   roldyoran/go-client  "/client"              32 seconds ago
Up 19 seconds  pr2so1-alto-ram-1789787906-22396
34ced917e16c   alpine     "sh -c 'while true; ..." 57 seconds ago
Up 50 seconds  pr2so1-alto-cpu-1789787881-4532
7ca09c2ab5bf   alpine     "sh -c 'while true; ..." About a minute ago
Up About a minute pr2so1-alto-cpu-1789787826-19482
32d65f7df680   alpine     "sleep 240"            About a minute ago
Up About a minute pr2so1-bajo-1789787825-18793
06e52913449f   alpine     "sleep 240"            2 minutes ago
Up 2 minutes   pr2so1-bajo-1789787761-29715
7e2aadbb8f90   alpine     "sleep 240"            3 minutes ago
Up 3 minutes   pr2so1-bajo-daemon-1789787730

```

5.5 Verificación de Valkey desde el host (redis-cli)

```
redis-cli -h localhost -p 6379 PING
redis-cli -h localhost -p 6379 KEYS '*'
redis-cli -h localhost -p 6379 GET pr2so1:ram:total_kb
redis-cli -h localhost -p 6379 ZREVRANGE pr2so1:ranking:ram 0 4 WITHSCORES
```



```
alejandro@alejandra-HP-Laptop-15-da0xxx: ~/Escritorio/201901657_LAB_...
pyecto2/daemon-go$ redis-cli -h localhost -p 6379 PING
redis-cli -h localhost -p 6379 KEYS '*'
redis-cli -h localhost -p 6379 GET pr2so1:ram:total_kb
redis-cli -h localhost -p 6379 ZREVRANGE pr2so1:ranking:ram 0 4 WITHSCORES
PONG
1) "pr2so1:ram:free_kb"
2) "pr2so1:ram:total_kb"
3) "pr2so1:ranking:ram"
4) "pr2so1:ranking:cpu"
5) "pr2so1:eliminados:total"
6) "pr2so1:ram:stream"
7) "pr2so1:eliminados:stream"
8) "pr2so1:ram:used_kb"
"12157188"
1) "pr2so1-alto-ram-1788647213-29401|0552135b93a1"
2) "10808"
3) "pr2so1-alto-ram-1788627749-30541|0788c6908d87"
4) "10808"
5) "pr2so1-alto-ram-1789787891-14311|818d7a2a6d11|pids:10746"
6) "10680"
7) "pr2so1-alto-ram-1789787821-11843|8ca5b9b0d287|pids:10091"
8) "10680"
9) "pr2so1-alto-ram-1789704604-2539|7f0c0af8a620|pids:7843"
10) "10680"
```

5.6 Restricciones de mínimos

```
crontab
docker ps --filter "label=pr2so1=managed" --format '{{.Names}}'
```

-l

```
alejandro@alejandro-HP-Laptop-15-da0xxx: ~/Escritorio/201901657_LAB_...
alejandro... x alejandro... x alejandro... x alejandro@... x alejandro@... x
alejandro@alejandro-HP-Laptop-15-da0xxx:~/Escritorio/201901657_LAB_S01_2S2026/pr
oyecto2/daemon-go$ crontab -l
docker ps --filter "label=pr2so1=managed" --format '{{.Names}}'
no crontab for alejandro
pr2so1-alto-ram-1789788109-7350
pr2so1-alto-cpu-1789788102-22677
pr2so1-bajo-1789788061-1862
pr2so1-bajo-daemon-1789788045
pr2so1-alto-cpu-1789788021-4659
pr2so1-bajo-daemon-1789788005
pr2so1-bajo-1789787941-1154
pr2so1-alto-cpu-1789787881-4532
alejandro@alejandro-HP-Laptop-15-da0xxx:~/Escritorio/201901657_LAB_S01_2S2026/pr
oyecto2/daemon-go$
```